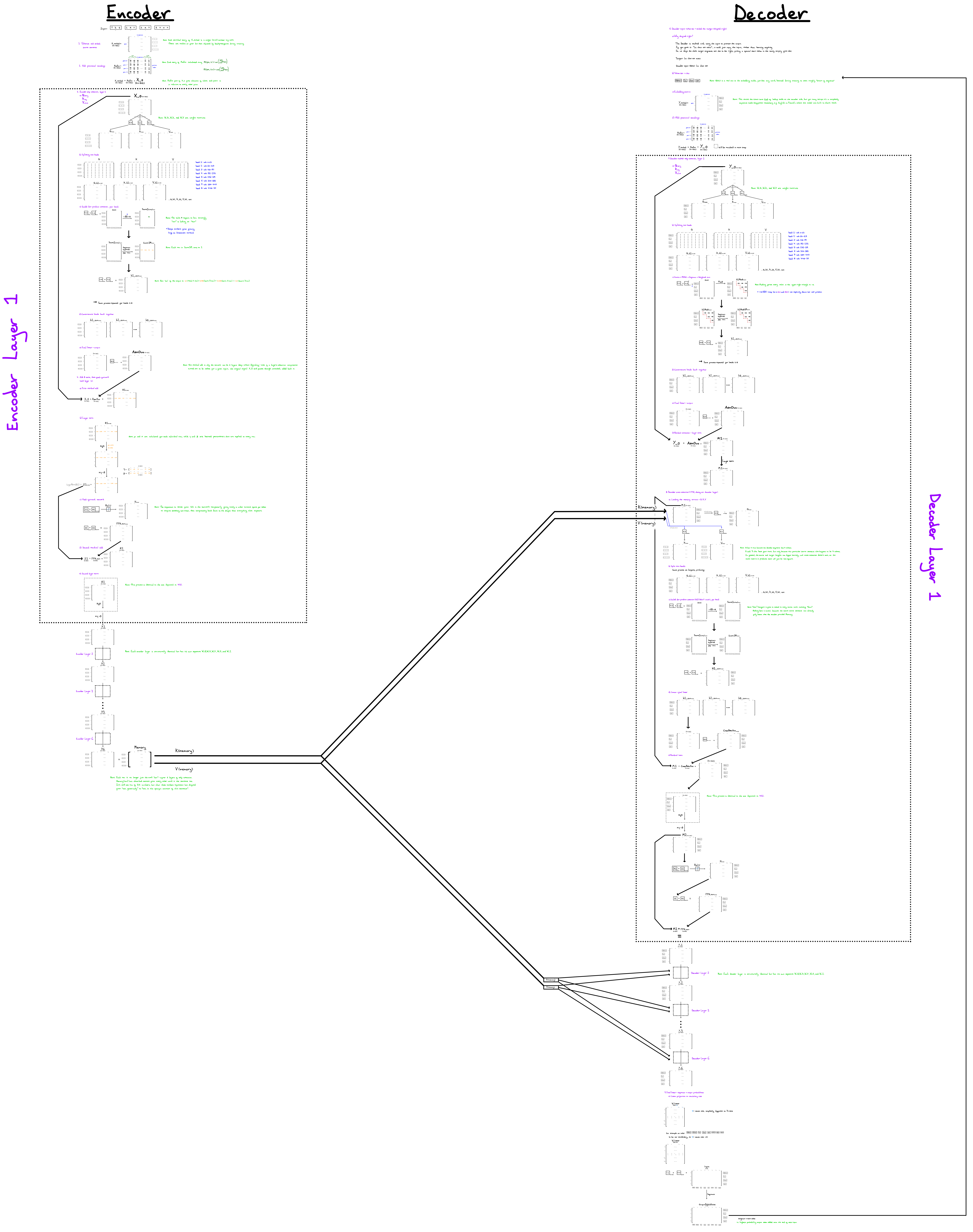
The cat sat down → Le chat est assis


Encoder
Input: The cat sat down
Encoder Layer 1
X_0 (4 × 512)
W_Q
W_K
W_V
multiplication
Q
K
V
Q_h1 (4×64)
K_h1 (4×64)
V_h1 (4×64)
, Q_h2, K_h2, V_h2, etc
ScoreInitial (4×4)
÷ √64 = 8
Softmax applied per row
ScoreSM (4×4)
h1_out (4×64)
h2_out (4×64)
h8_out (4×64)
AttnOut (4 × 512)
W_O
R1 (4×512)
μ=1.223
σ=1.731
$\frac{x-\mu}{\sigma}$
$x\gamma+\beta$
LayerNorm(R1) = Z1 (4×512)
ReLU
H
FFN_out
R2 (4 × 512)
X1 (4 × 512)
Encoder Layer 2
X2 (4 × 512)
Encoder Layer 3
X5 (4 × 512)
Encoder Layer 6
X6 (4 × 512)
Memory (4 × 512)
K(memory)
V(memory)
Decoder
Decoder Layer 1
Y_0 (4 × 512)
Mask
h1Mask (4×4)
h1MaskSM (4×4)
M1 (4 × 512)
Layer norm
CrossAttnOut
M2 (4 × 512)
Y_1 (4 × 512)
Decoder Layer 2
Y_2 (4 × 512)
Decoder Layer 3
Y_5 (4 × 512)
Decoder Layer 6
Y_6 (4 × 512)
W_linear (512 × V)
W_linear (512 × 8)
Logits (4 × 8)
Softmax
OutputProbabilities (4 × 8)
argmax → next token

Encoder

Input: The cat sat down

1. Tokenize and embed source sentence

$X\_embed =$ (4 × 512)

Note: Each individual entry of X_embed is a weight (small number e.g. 0.09). These are random at first but then adjusted by backpropagation during training.

2. Add positional encodings

$PosEnc =$ (4 × 512)

Note: Each entry of PosEnc calculated using $PE(pos, 2i) = \sin\left(\frac{pos}{1000^{(2i/512)}}\right)$

$PE(pos, 2i+1) = \cos\left(\frac{pos}{1000^{(2i/512)}}\right)$

$X\_embed_{(4\times512)} + PosEnc_{(4\times512)} = X\_0_{(4\times512)}$

Note: PosEnc part of X_0 gives indication of where each point is in relation to every other point.

3. Encoder self-attention, layer 1

a) Query Key Value

Note: W_Q, W_K, and W_V are weight matrices

b) Splitting into heads

head 1: cols 0-63
head 2: cols 64-127
head 3: cols 128-191
head 4: cols 192-255
head 5: cols 256-319
head 6: cols 320-383
head 7: cols 384-447
head 8: cols 448-511

c) Scaled dot-product attention, per head

Note: The value * refers to how strongly "cat" is looking at "sat"

* keeps numbers from growing huge as dimensions increase

Note: Each row in ScoreSM sums to 1

Note: Row 'cat' of the output is 0.1(The's V row) + 0.55(cat's V row) + 0.25(sat's V row) + 0.1(down's V row)

→ Same process repeated for heads 2-8

d) Concatenate heads back together

e) Final linear + output

Note: This residual add is why the network can be 6 layers deep without degrading - even if a layer's attention computation turned out to be useless for a given input, the original signal X_0 still passes through untouched, added back in.

4. Add & norm, then feed-forward (still layer 1)

a) First residual add

$X\_0_{(4\times512)} + AttnOut_{(4\times512)} =$

b) Layer norm

Note: μ and σ are calculated for each individual row, while γ and β are learned parameters that are applied to every row.

c) Feed-forward network

Note: The expansion to 2048 from 512 is the network temporarily giving itself a wider scratch space per token to compute something non-linear, then compressing back down to the shape that everything else expects.

d) Second residual add

$Z1_{(4\times512)} + FFN\_out_{(4\times512)} =$

e) Second layer norm

Note: This process is identical to the one depicted in 4(b).

Note: Each encoder layer is structurally identical but has its own separate W_Q, W_K, W_V, W_O, and W_1.

Note: Each row is no longer just the word "cat" - after 6 layers of self-attention, Memory[cat] has absorbed context from every other word in the sentence too. It's still one row of 512 numbers, but what those numbers represent has shifted from "cat, generically" to "cat, in the specific context of this sentence".

Decoder

6. Decoder input: tokenize + embed the target (shifted right)

a) Why shifted right?

The decoder is tasked with using the input to protect the output.
If you gave it "Le chat est assis", it could just copy the input, rather than learning anything.
So, we shift the whole target sequence one slot to the right, putting a special start token in the newly-empty first slot.

Target: Le chat est assis

Decoder input: <BOS> Le chat est

b) Tokenize + tiles

<BOS> Le chat est

Note: <BOS> is a real row in the embedding table, just like any word, learned during training to mean roughly "start of sequence"

c) Embedding matrix

$Y\_embed =$ (4 × 512)

Note: This reuses the exact same kind of lookup table as the encoder side, but for many setups it's a completely seperate table (different vocabulary e.g. English vs French), unless the model was built to share vocab.

d) Add positional encodings

$Y\_embed_{(4\times512)} + PosEnc_{(4\times512)} = Y\_0_{(4\times512)}$

Will be masked in next step

7. Decoder masked self-attention, layer 1

a) Query Key Value

Note: W_Q, W_K, and W_V are weight matrices

b) Splitting into heads

head 1: cols 0-63
head 2: cols 64-127
head 3: cols 128-191
head 4: cols 192-255
head 5: cols 256-319
head 6: cols 320-383
head 7: cols 384-447
head 8: cols 448-511

c) Scores → MASK → Softmax → Weighted sum

Note: Masking forces every value in the upper-right triangle to -∞.

* ÷ 8 (√64) step (as in 3(c) and 8(c)) not explicitly shown but still present.

→ Same process repeated for heads 2-8

d) Concatenate heads back together

e) Final linear + output

d) Residual connection + layer norm

$Y\_0_{(4\times512)} + AttnOut_{(4\times512)} =$

8. Decoder cross-attention (+ FFN, closing out decoder layer)

a) Landing the memory arrows + Q, K, V

Note: Q has 4 rows because the decoder sequence has 4 tokens. K and V also have four rows, but only because this particular source sentence also happens to be 4 tokens. In general, the source and target lengths can differ entirely, and cross-attention doesn't care, as the score matrix it produces next will just be non-square.

b) Split into heads

Same process as before, producing:

c) Scaled dot-product attention (WITHOUT mask), per head

Note: "chat" (target) is free to attend to every source word, including "down". Nothing here is causal, because the entire source sentence was already fully known when the encoder provided Memory.

d) Concat + final linear

e) Residual + norm

$M1_{(4\times512)} + CrossAttnOut_{(4\times512)} =$

Note: This process is identical to the one depicted in 4(b).

$M2_{(4\times512)} + FFN\_out_{(4\times512)} =$

Note: Each decoder layer is structurally identical but has its own separate W_Q, W_K, W_V, W_O, and W_1.

9. Final linear + softmax → output probabilities

a) Linear projection to vocabulary size

V = vocab size, completely different to V = value

For example we take: <BOS> <EOS> Le chat est assis sur tapis to be our vocabulary, so V = vocab size = 8

argmax → next token

i.e. highest probability output token added onto the end of next input